

PROJECT REPORT

Implement Client Script & UI Policy (Incident)

TEAM MEMBERS

1. Ruthrash B
2. Gokulnath M
3. Vikash S
4. Monikha Shree K
5. Lakshitha MC

1. INTRODUCTION

1.1 Project Overview

Incident management is a core function of ServiceNow IT Service Management. When users create or update incidents, the form should guide them to enter complete and valid information. This project focuses on implementing Client Scripts and UI Policies on the Incident form to improve data validation, field behavior, usability, and consistency.

Client Scripts execute client-side JavaScript to perform actions such as setting field values, validating user input, making fields mandatory, displaying messages, and responding to field changes. UI Policies control the behavior of form fields without requiring scripting for common conditions, such as making a field mandatory, visible, or read-only.

1.2 Purpose

- Improve the Incident form user experience.
- Automatically control Incident fields based on conditions.
- Validate important information before an Incident is submitted.
- Make relevant fields mandatory when required.
- Show or hide fields according to Incident conditions.
- Make selected fields read-only when appropriate.
- Reduce incomplete or incorrect Incident records.
- Improve consistency in Incident data entry.

2. IDEATION PHASE

2.1 Problem Statement

Incident records are often created with incomplete, inconsistent, or incorrect information. Users may not know which fields are required for a particular incident state, category, or priority. Without appropriate form controls, support teams may spend additional time correcting records or requesting missing information.

- Important Incident fields may be left empty.

- Users may enter invalid or incomplete information.
- Fields may not dynamically respond to Incident conditions.
- Unnecessary fields can make the form difficult to use.
- Support teams may need to correct Incident records manually.
- Inconsistent data can affect Incident analysis and reporting.

Proposed Problem Statement: How can ServiceNow Client Scripts and UI Policies be implemented on the Incident form to validate user input and dynamically control field visibility, mandatory status, and read-only behavior?

2.2 Empathy Map Canvas

Category	Incident User	IT Support Team
Says	I need to report my issue quickly; I want the form to be simple.	We need complete and accurate Incident information.
Thinks	Which fields do I need to fill? Why is this field required?	Can form validation reduce incomplete records?
Does	Creates, updates, and submits Incident records.	Reviews Incidents, updates fields, and resolves issues.
Feels	Confused when unnecessary fields appear or validation is unclear.	Frustrated by missing information; satisfied with consistent records.

2.3 Brainstorming

- Make relevant Incident fields mandatory based on conditions.
- Show or hide fields dynamically.
- Make selected fields read-only after specific conditions.
- Validate Incident field values using Client Scripts.
- Display helpful messages when user input is invalid.
- Set default or derived values automatically.
- Use UI Policies for simple field behavior.
- Use Client Scripts for client-side validation and dynamic logic.
- Test the form using different Incident scenarios.

Selected Idea: Implement Client Script and UI Policy rules on the ServiceNow Incident form to provide dynamic field behavior and client-side validation.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

Stage	User Action	System Action	User Experience
Incident Creation	Opens Incident form	Displays Incident fields	Simple
Data Entry	Enters issue details	Client Script/UI Policy responds to conditions	Guided
Validation	Submits Incident	Validates	Clear

		required/input values	
Update	Changes priority/category/state	Field behavior changes dynamically	Consistent
Save	Saves Incident	Stores valid information	Reliable
Resolution	Updates resolution details	Relevant fields become mandatory/read-only as configured	Controlled

3.2 Solution Requirement

Functional Requirements

- Allow users to create and update Incident records.
- Configure UI Policies for Incident field visibility, mandatory status, and read-only behavior.
- Configure Client Scripts for client-side validation and dynamic field actions.
- Display validation messages when required conditions are not satisfied.
- Automatically set or clear field values when appropriate.
- Prevent submission when configured validation rules fail.
- Support Incident form behavior for different categories, priorities, and states.
- Maintain consistent Incident data entry.

Non-Functional Requirements

- Reliability: Form rules should execute consistently.
- Performance: Client-side actions should not unnecessarily slow the form.
- Security: Client-side controls should not be treated as the only security mechanism.
- Maintainability: Administrators should be able to understand and update the rules.
- Usability: Messages and field behavior should be clear to users.
- Scalability: The solution should support additional Incident rules as requirements grow.

3.3 Data Flow Diagram

User → Incident Form → UI Policy / Client Script → Field Validation & Dynamic Behavior → Valid Incident Data → Save/Submit → Incident Record

Detailed Process Flow:

1. User opens the Incident form.
2. User enters Incident details.
3. UI Policy checks configured conditions and changes field behavior.
4. Client Script responds to form events such as onLoad, onChange, or onSubmit.
5. The system validates required or conditional information.
6. If validation fails, an appropriate message is displayed and submission may be stopped.
7. If validation succeeds, the Incident is submitted or updated.
8. The Incident record is stored with consistent information.

3.4 Technology Stack

Technology	Purpose
ServiceNow	IT service management platform
Incident Management	Create and manage Incident records
Client Scripts	Client-side validation and dynamic form behavior
UI Policies	Control field visibility, mandatory status, and read-only behavior
JavaScript	Programming language used in Client Scripts
ServiceNow Tables	Store Incident information
ServiceNow UI	User interaction and administration

4. PROJECT DESIGN

4.1 Problem Solution Fit

Problem	Proposed Solution
Incomplete Incident information	UI Policies make relevant fields mandatory
Incorrect user input	Client Scripts validate input values
Unnecessary fields visible	UI Policies show/hide fields based on conditions
Fields changed when they should not be edited	UI Policies make fields read-only
Unclear form behavior	Dynamic field behavior guides users
Repeated manual validation	Client-side validation reduces avoidable errors
Inconsistent Incident records	Standardized form rules improve data consistency

The proposed solution directly addresses common Incident form usability and data-quality problems by combining UI Policies for declarative field control with Client Scripts for client-side logic and validation.

4.2 Proposed Solution

9. Step 1 – Create or open the Incident form in ServiceNow.
10. Step 2 – Identify fields that require conditional visibility, mandatory status, or read-only behavior.
11. Step 3 – Configure UI Policies for simple field behavior based on Incident conditions.
12. Step 4 – Create Client Scripts for onLoad, onChange, or onSubmit requirements where scripting is necessary.
13. Step 5 – Validate user-entered values and display clear messages when values are invalid.
14. Step 6 – Test different Incident categories, priorities, states, and user inputs.
15. Step 7 – Correct configuration issues and verify that the rules do not conflict.
16. Step 8 – Deploy the completed configuration for normal Incident processing.

4.3 Solution Architecture

User → ServiceNow Incident Form → UI Policy / Client Script → Field Behavior & Validation → Incident Record → IT Support Team

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Phase	Activity	Duration
Phase 1	Understand Incident management and form requirements	Day 1
Phase 2	Identify fields and business conditions	Day 2
Phase 3	Design UI Policy rules	Day 3
Phase 4	Configure Client Scripts	Day 4–5
Phase 5	Configure Incident form behavior	Day 6
Phase 6	Test field visibility, mandatory and read-only rules	Day 7
Phase 7	Test Client Script validation	Day 8
Phase 8	Fix conflicts and refine usability	Day 9
Phase 9	Functional testing	Day 10
Phase 10	Documentation and final demonstration	Day 11–12

Project Deliverables

- Configured Incident form.
- UI Policy configurations.
- Client Script configurations.
- Client-side validation rules.
- Test cases and testing results.
- Screenshots of Incident form behavior.
- Project documentation.

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Test Case	Input / Action	Expected Result	Result
1 – Incident Form Load	Open a new Incident	Configured onLoad behavior is applied	Form loads correctly
2 – Field Visibility	Change configured Incident condition	Relevant field is shown/hidden	Behavior works as configured
3 – Mandatory Field	Select a condition requiring a field	Field becomes mandatory	Mandatory rule works
4 – Read-Only Field	Trigger configured condition	Selected field becomes read-only	Read-only rule works
5 – Invalid Input	Enter an invalid value	Client Script displays validation message	Invalid input is handled
6 – Valid Input	Enter valid information	Incident can be	Submission succeeds

		submitted/updated	
7 – OnSubmit Validation	Submit Incident with missing required information	Submission is prevented and message displayed	Validation works
8 – Rule Conflict Check	Test multiple conditions together	Rules behave consistently without unwanted conflicts	Configuration verified

7. RESULTS

7.1 Output Screenshots

- ServiceNow Login/Home Page.
- Incident module.
- New Incident form.
- Configured UI Policy.
- UI Policy condition and actions.
- Client Script configuration.
- Client Script code.
- Incident form showing a mandatory field.
- Incident form showing a hidden field.
- Incident form showing a read-only field.
- Validation message for invalid input.
- Successfully submitted Incident.

Expected Output: The final system demonstrates a controlled and user-friendly Incident form in which UI Policies dynamically manage field behavior and Client Scripts validate user input. The result is more consistent Incident data with fewer avoidable form-entry errors.

8. ADVANTAGES & DISADVANTAGES

Advantages

- Improves Incident form usability.
- Reduces incomplete Incident records.
- Provides immediate client-side validation.
- Automatically controls field visibility.
- Makes relevant fields mandatory when required.
- Prevents unwanted changes through read-only controls.
- Reduces repetitive manual correction by support teams.
- Improves consistency of Incident data.
- Can be configured for different Incident conditions.

Disadvantages

- Complex Client Scripts can be difficult to maintain.

- Too many UI Policies may make form behavior difficult to understand.
- Client-side validation does not replace server-side security or validation.
- Changes to Incident requirements may require configuration updates.
- Incorrect scripts can affect the user experience or form behavior.
- Poorly designed rules may conflict with other Incident configurations.

9. CONCLUSION

The project “Implement Client Script & UI Policy (Incident)” demonstrates how ServiceNow Client Scripts and UI Policies can be used to improve Incident form behavior. UI Policies provide configurable control over field visibility, mandatory status, and read-only behavior, while Client Scripts provide client-side validation and dynamic actions.

By applying these configurations to the Incident form, the system guides users to provide appropriate information, reduces incomplete or invalid entries, and improves the consistency and usability of Incident management. The project demonstrates practical ServiceNow configuration skills that support efficient IT service management.

10. FUTURE SCOPE

- Add more condition-based UI Policies for different Incident categories.
- Introduce advanced validation for additional Incident fields.
- Integrate form behavior with other ServiceNow workflows and automation.
- Create standardized validation rules for different support teams.
- Improve user guidance with contextual messages and help text.
- Use ServiceNow reporting to monitor Incident data quality.
- Extend validation requirements to additional ITSM forms.
- Review and optimize Client Scripts for better maintainability and performance.

11. APPENDIX

Source Code

The project uses ServiceNow configuration and JavaScript-based Client Scripts rather than a standalone application. The appendix can include screenshots of UI Policies, UI Policy Actions, Client Script configurations, Client Script code, Incident form behavior, and testing results.

Dataset Link

No external dataset is required for the basic implementation. The project uses ServiceNow Incident records and the fields available on the Incident table.

GitHub & Project Demo Link

GitHub: Add the project repository link if configuration documentation or supporting files are maintained in GitHub. Project Demo: Add the ServiceNow instance/demo URL if available.